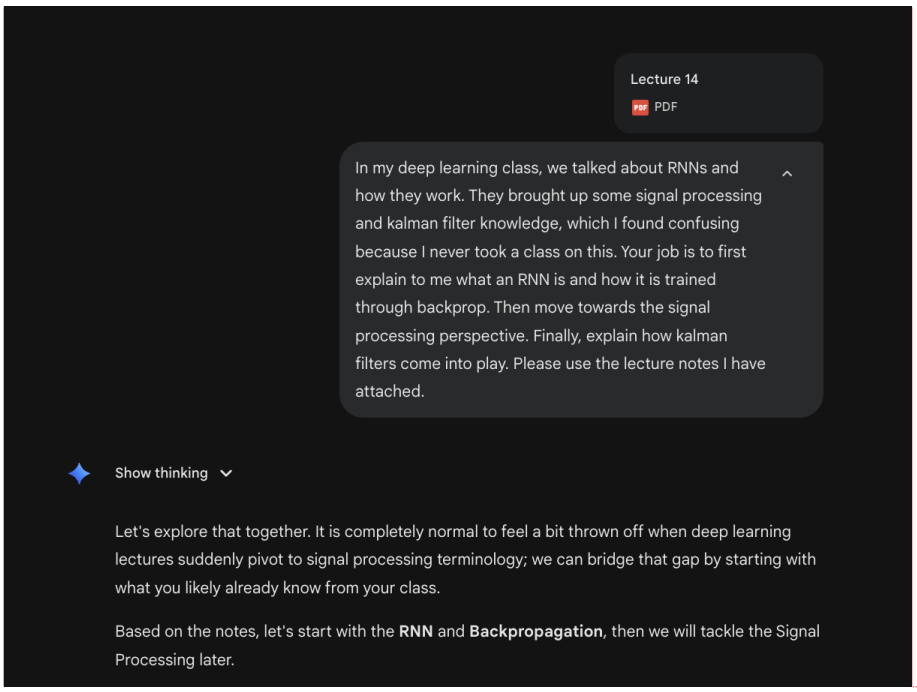



**Commented [1]:** I included that I have taken classes that covered signal processing and Kalman filters so Gemini can take this into account.

**1. What is an RNN?**

In your previous lectures, you likely used **CNNs** for images. The notes make a key distinction here: while CNNs share weights across *space* (the image dimensions), **RNNs (Recurrent Neural Networks)** share weights across *time*.

This is designed for "sequential data" like language, speech, or control systems.

The core feature of an RNN is the **Hidden State** ($h_t$). Think of this as the network's "memory." At every time step $t$, the network looks at:

1. The new input ($u_{t+1}$).
2. The *previous* hidden state ($h_t$).

The notes provide this linear update equation (before adding nonlinearities):

$$h_{t+1} = W_h h_t + B u_{t+1}$$

This recursive update is what allows the network to carry information forward indefinitely.

**2. Training via Backpropagation**

To train this, we "unroll" the network over time. Instead of thinking of it as one loop, imagine making copies of the network for each time step ($t, t+1, t+2 \ldots$) and connecting them in a line .

- **Forward Pass:** We compute the states moving forward in time.
- **Backward Pass:** We calculate the loss at the "Loss Layer" (usually at the end or at every step). Then, we propagate the error gradients *backwards* through time, from the last time step back to the first.

**Commented [2]:** I really like how Gemini compared CNNs, something I have more knowledge about, to RNNs, something I want to learn more about.

**The Challenge:** The notes highlight a major issue here. Because the path backwards through time can be very long, the gradients can "die out" (vanish) before they reach the beginning of the sequence. The notes mention that using "saturating nonlinearities" (like tanh or sigmoid) makes this problem even worse.

---

To make sure we have the foundation solid before switching to the Signal Processing perspective:

Looking at the update equation $h_{t+1} = W_h h_t + B u_{t+1}$, if $W_h$ is a small number (less than 1) and we multiply it by itself many times over a long sequence, what do you think happens to the value of the hidden state (the "memory") from the start of the sequence?

I had a question before we move onto to the question. How is the input actually passed into the RNN, since it is sequential.

Show thinking

That is a fantastic question to ask before we go further! It really helps to visualize the "mechanics" of how the data flows.

In a standard Feedforward Neural Network (like an MLP), you might feed an entire image or a whole set of features all at once. In an **RNN**, we process the input **step-by-step** over time.

**Commented [3]:** I like how Gemini brought up the issue of vanishing gradients without even asking it to.

**Commented [4]:** I was a bit confused about how the input is fed into the RNN. Even though I knew it was sequential data, I needed a proper example.

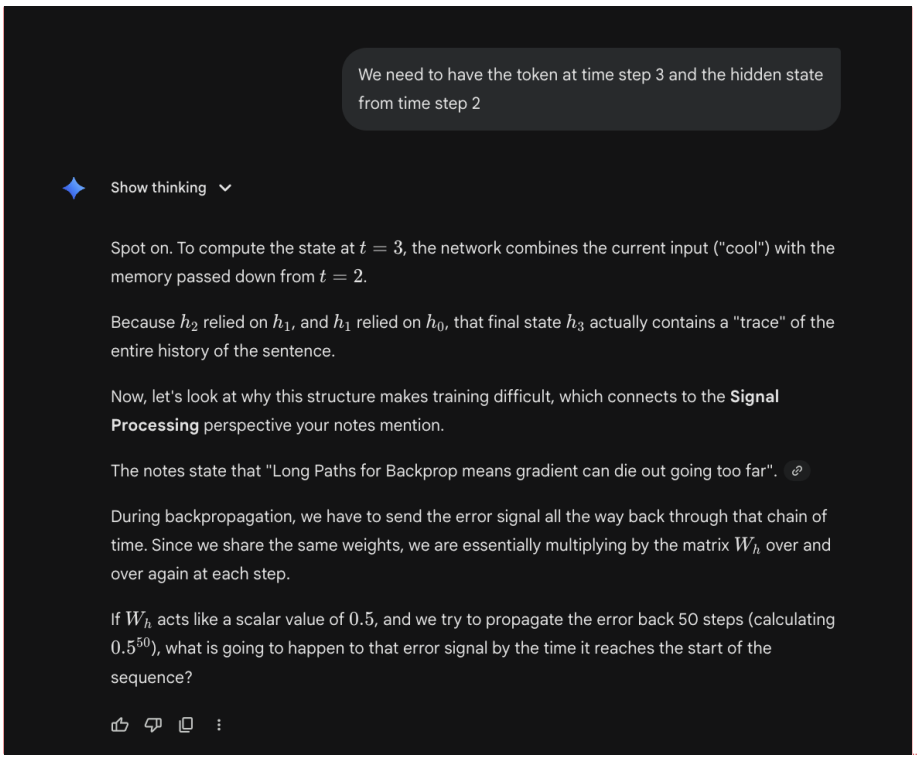
Here is the breakdown:

1. **The Sequence:** Imagine your input is a sentence like "AI is cool." We break this down into individual distinct time steps (often called tokens).
   - $t = 1$: "AI"
   - $t = 2$: "is"
   - $t = 3$: "cool"
2. **The "Unrolled" Process:** Even though we often draw an RNN as a single box with a loop, we can "unroll" it to see what happens during training.
   - At **Time 1**: The network takes the vector for "AI" ($u_1$) and the initial hidden state (usually zeros). It calculates the new hidden state $h_1$.
   - At **Time 2**: It takes the vector for "is" ($u_2$) **AND** the hidden state from the previous step ( $h_1$). It uses these to calculate $h_2$.
   - At **Time 3**: It takes "cool" ($u_3$) and $h_2$ to calculate $h_3$.

The key insight from your notes is **Weight Sharing**. Even though we process three different words at three different times, we use the **exact same** weight matrices ($W_h$ and $B$) at every single step.

To verify this concept, if we were processing that sentence ("AI is cool") and we are currently at the step calculating the hidden state for "cool" ($h_3$), what two specific pieces of information does the network need to have ready?

**Commented [5]:** This example was super useful to solidifying my understanding of how hidden states are calculated in an RNN. I also liked how Gemini asked me a question to follow up my understanding.




**Commented [6]:** I liked how Gemini connected to the signal processing perspective and how the signal can vanish by the time is it propagated back to the start of the sequence.

That error signal will vanish by the time it reaches the start

Show thinking

Exactly. That is the famous "Vanishing Gradient" problem. If the weights were large (e.g., 2.0), you would have the opposite problem: "Exploding Gradients" ($2^{50}$ is a massive number).

This behavior—where a signal circulates and multiplies over time—is the perfect bridge to the **Signal Processing** perspective mentioned in your notes.

**2. The Signal Processing Perspective**

In Signal Processing, we classify filters based on how long an input continues to affect the output. Your lecture notes make a specific comparison here:

| Neural Net | Signal Processing Equivalent | Behavior |
|---|---|---|
| **CNN** | **FIR** (Finite Impulse Response) | "Memory" is limited to the kernel size (e.g., 3 pixels wide). The effect of an input dies immediately after the window passes. |
| **RNN** | **IIR** (Infinite Impulse Response) | "Memory" can theoretically last forever because the output feeds back into the input. |

Export to Sheets

**Commented [7]:** After Gemini walked through the vanishing gradient problem, I understood the IIR filter equivalent for an RNN better

**Why does this matter?**

The lecture is trying to tell you that an RNN is essentially a digital **IIR Filter**.

- **FIR (Finite):** Think of a CNN. It looks at a specific patch of data. Once it moves past that patch, it forgets it. It is stable but has limited context.
- **IIR (Infinite):** Think of an RNN. Because $h_{t+1}$ depends on $h_t$, which depended on $h_{t-1}$, a kick to the system at $t = 0$ can essentially "echo" inside the loop forever.

The notes explicitly define the RNN as a dynamical system update, which is identical to how engineers write IIR filters:

$$h_{t+1} = W_h h_t + B u_{t+1}$$

- $u$: The Input signal (new data)
- $h$: The Hidden State (the "echo" or memory inside the filter)

This perspective helps explain the training difficulty: **IIR filters are notoriously hard to keep stable.** If the feedback gain ($W_h$) is slightly too high, the energy explodes. If it's slightly too low, the signal vanishes.

---

**3. The Kalman Filter**

Now, the lecture introduces the **Kalman Filter** to formalize this "Linear Dynamical System" idea.

If you strip away the complex activation functions (like tanh or sigmoid) from an RNN, what you are left with is purely linear equations. This linear version is exactly what a Kalman Filter solves.

**Commented [8]:** I think the introduction to Kalman filters was a bit sudden here. Gemini didn't actually explain what the Kalman filter is before introducing it in the perspective of an RNN.

**The "True" Linear Process**

The notes describe a world where there is a hidden "true" state ($x$) that changes linearly over time, but we can't see it directly. We only see noisy measurements ($y$):

1. **State Update:** $x_{t+1} = Ax_t + \dots$ (The true state evolves)
2. **Observation:** $y_{t+1} = Cx_{t+1} + \dots$ (We see a noisy version of it)

**The Goal**

The Kalman Filter is the mathematically "optimal" algorithm for guessing that hidden state $x$ based on the noisy $y$ inputs.

**The Connection to RNNs**

The lecture makes a profound point here: **An RNN is essentially a learned, non-linear approximation of a Kalman Filter.**

- **Kalman Filter:** You usually need to *know* the matrices $A$, $B$, and $C$ beforehand (physics equations) to predict the state optimally.
- **RNN:** We *don't* know the physics of language or speech. So, we treat the matrices as "learnable parameters" (Weights) and use Backpropagation to figure them out from data.

The lecture suggests that if we don't know the physics (matrices $A, B, C \dots$), we can treat the RNN as a "black box" that learns to mimic the behavior of a Kalman Filter just by looking at the data traces.

**Commented [9]:** Interesting...when I first watched the lecture about RNNs, I didn't really think about the distinction between the true state update and observation, each which has its own equation describing it.

**To summarize the flow:**

1. **RNN:** A neural net with memory (loops).
2. **Signal Processing:** That loop makes it an **IIR Filter** (infinite memory, prone to instability).
3. **Kalman Filter:** The "perfect" mathematical version of this loop for linear problems. The RNN tries to learn a similar tracking capability from raw data without knowing the underlying equations.

Does that distinction between the "perfect" math of the Kalman Filter vs. the "learned" approximation of the RNN make sense?

I guess it would help introducing what a Kalyan filter actually is...since I haven't seen it before.

Show thinking

That is a completely fair point. The lecture likely assumed everyone had taken a control theory class, which is often not the case!

Here is the "missing lecture" on the **Kalman Filter** to help you make sense of that comparison.

**Commented [10]:** As mentioned before, I wanted an actual definition of the Kalman filter before understanding it in the context of RNNs.

**1. The Intuition: The GPS in a Tunnel**

Imagine you are driving a car into a tunnel where your GPS signal gets spotty and noisy. The Kalman Filter is the algorithm your car's navigation system uses to figure out where you *actually* are.

It works by combining two sources of information, neither of which is perfect:

1. **The Prediction (The "Physics" Model):** You know your speed and direction. Even without GPS, you can guess where you will be 1 second from now. (e.g., "I was at meter 100, moving at 10 m/s, so I should be at meter 110.")
2. **The Measurement (The Noisy Sensor):** The GPS sends a ping saying you are at meter 112. It's noisy and jumps around.

**The Kalman Filter** is a mathematical trick to merge these two.

- If you know your GPS is terrible (high noise), you trust your **Prediction** more.
- If you know your driving speed is erratic (unreliable physics), you trust the **Measurement** more.

**2. The Math (The "A, B, C" Matrices)**

The lecture notes mention "linear dynamical systems" and matrices like $A$, $B$, and $C$. This is just the formal way of writing the car example above.

In a Kalman Filter, we assume the world follows these linear rules:

**Rule 1: The Transition (How the state evolves)**

$$x_{t+1} = Ax_t + Bu_{t+1}$$

**Commented [11]:** This analogy is honestly pretty good in connecting to Kalman filters but maybe it is too unrelated...

- $x$ **(Hidden State):** The "True" position of the car (which we can't see directly).
- $A$ **matrix:** The physics. (e.g., "New position = Old position + Velocity").
- $u$ **(Input):** Did you step on the gas? (Control input).

**Rule 2: The Observation (What we see)**

$$y_{t+1} = Cx_{t+1} + \text{Noise}$$

- $y$ **(Observation):** The noisy number on the GPS screen.
- $C$ **matrix:** How the hidden state maps to the sensor (e.g., The GPS measures position, but maybe not velocity).

**3. The "Loop" (Predict & Update)**

The Kalman Filter runs in a loop, exactly like an RNN:

1. **Predict Step:** Use Rule 1 ($A$ and $B$) to guess where the car is now, based on where it was a second ago.
2. **Update Step:** Look at the new measurement $y$. Compare it to your guess.
   - *Difference = (Measurement - Guess)*
   - Multiply this difference by a calculated "Gain" (how much we trust the sensor).
   - Correct your current position estimate.

**4. Connecting it back to RNNs**

Now, look at the RNN equation from your notes again:

$$h_{t+1} = W_h h_t + W_x x_{t+1}$$

Do you see the similarity to Rule 1 of the Kalman Filter?

- **RNN:** $h_{t+1} = W_h h_t + \dots$
- **Kalman:** $x_{t+1} = A x_t + \dots$

**The Big Takeaway:** The lecture is saying that an RNN is trying to be a "Learned Kalman Filter."

- In a **Kalman Filter**, an engineer sits down and *calculates* the matrices $A$ and $B$ using physics equations (Newton's laws).
- In an **RNN**, we don't know the physics (we don't know the "Newton's laws" of English grammar). So, we use **Backpropagation** to *learn* the matrices $W_h$ and $W_x$ from the data, hoping they will act just like $A$ and $B$ to track the "state" of the sentence.

Does that help clarify why they suddenly started talking about $A, B, C$ matrices and "State Estimation"?

Ok that makes more sense. I'm a bit confused about the picture about RNNs, where the vertical direction represents layers and the horizontal direction represents time. Can you explain it to be more clearly?

**Commented [12]:** This helped me understand how the RNNs can be thought of learned Kalman filters, in which we actually need to learn the weight matrices through backprop.

It can be tricky because when we draw simple RNNs, we usually just draw a loop. But the "Vertical vs. Horizontal" picture you are looking at (likely on **Page 3** of your notes) describes a **Deep RNN** (also known as a Stacked RNN).

Think of this structure as a literal grid or a lattice. Here is how to break down the two axes:

**1. The Vertical Axis: "Thinking Deeply" (Layers)**

**Direction:** Bottom $\rightarrow$ Top **Function:** Feature Extraction (Complexity)

This is the standard Neural Network direction you are already used to.

- **Bottom:** You feed raw data in (e.g., the word "Kalman").
- **Middle:** The network processes it through multiple layers (Layer 1 $\rightarrow$ Layer 2 $\rightarrow$ Layer 3).
- **Top:** You get a prediction out (e.g., probability of the next word).

In the notes, this is labeled as "Vertical '1x1' MLPs" or "Vertical Residual Connections". Just like in a CNN, adding more vertical layers allows the network to learn more complex, abstract features about the *current* input.

**2. The Horizontal Axis: "Thinking Forward" (Time)**

**Direction:** Left $\rightarrow$ Right **Function:** Memory (Context)

This is the unique RNN feature.

- **Left:** The past (Time $t$).
- **Right:** The future (Time $t+1$).

Information flows horizontally to carry **context**. If the network is reading a sentence, the horizontal arrow carries the memory of the word it read previously.

**Commented [13]:** I was having trouble understanding the picture of RNNs with the vertical and horizontal axes, so I asked Gemini to explain this. This is a bit unrelated to the whole signal processing perspective but still useful nonetheless.

**3. Putting them together: The Grid**

When you combine these, you get a grid of computations. Look at a single box in the middle of that diagram (say, **Layer 2** at **Time 2**). That box needs two inputs to do its job:

1. **From Below (Vertical):** The processed data from **Layer 1** at the *same* time. (What is the current word?)
2. **From the Left (Horizontal):** The hidden state from **Layer 2** at the *previous* time. (What was the context from the previous word?)

**Why do we do this?**

- **Horizontal Depth:** Allows us to remember things from long ago (Long-term memory).
- **Vertical Depth:** Allows us to understand complex relationships in the current moment (Hierarchical processing).

The notes highlight a "Radical Approach" related to this: **Parallelism**.

- **Vertical steps** can be parallelized to some degree (pipelining), but...
- **Horizontal steps** generally cannot. You strictly cannot calculate Time 2 until you have finished Time 1. This "Sequential Causality" is the major bottleneck of RNNs that the lecture mentions later.

Sounds good. Can you walk through a simple example input into an RNN so I can put everything together.

Show thinking

Let's walk through a concrete example using the sentence **"AI learns"**. We will use a "Single-Layer" RNN first to keep it simple, and then I will point out where the "Vertical" (Deep) part would fit in.

To make this easy to visualize, imagine our RNN has a very simple "brain" (Hidden State) that can only hold **2 numbers**.

**The Setup**

- **Input ($u$):** The words "AI" and "learns". We turn these into vectors (numbers).
  - `AI` = $[1, 0]$
  - `learns` = $[0, 1]$
- **The State ($h$):** Starts at zero: $[0, 0]$.
- **The Weights ($W_h$ and $B$):** The "learned physics" of the system.
  - $B$ (Input Processor): Decides how much the *new word* matters.
  - $W_h$ (Memory Processor): Decides how much the *past* matters.

---

**Step 1: Time $t = 1$ (Input: "AI")**

We feed the first word into the network.

1. **Horizontal Input (Past):** The hidden state is empty ($h_0 = [0, 0]$).
2. **Vertical Input (Present):** The current word is "AI" ($u_1 = [1, 0]$).
3. **The Update Calculation:** The network combines them using the formula from your notes: $h_1 = W_h h_0 + B u_1$.

**Status Check:**

- **Memory:** The RNN now "remembers" it has seen the concept of "AI".
- **Kalman Connection:** This is like the **Prediction Step**. We updated our belief state based on the first observation.

---

**Step 2: Time $t = 2$ (Input: "learns")**

Now we feed the second word. This is where the magic happens.

1. **Horizontal Input (Past):** We *don't* start from zero. We pull the state from Step 1 ($h_1 = [0.5, 0.5]$).
2. **Vertical Input (Present):** The current word is "learns" ($u_2 = [0, 1]$).
3. **The Update Calculation:**

   $h_2 = W_h h_1 + B u_2$.
   - The network takes the memory of "AI" $(0.5, 0.5)$ AND the new input "learns" $(0, 1)$.
   - It mixes them together.
   - **New State ($h_2$):** Maybe the result is $[0.8, -0.2]$.

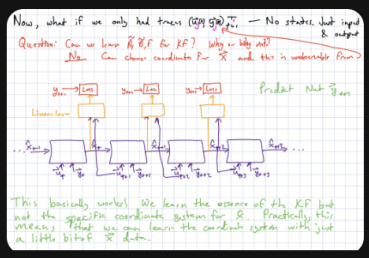
**Status Check:**

- **Memory:** The vector $[0.8, -0.2]$ is a specific numerical blend representing the *entire phrase* "AI learns".
- **Vertical vs. Horizontal:**
  - **Horizontal:** The info $[0.5, 0.5]$ traveled *sideways* from Time 1 to Time 2.
  - **Vertical:** The word "learns" traveled *upwards* into the calculation at Time 2.

**Tying it all together**

1. **The "Grid" (Deep RNNs):** If this were a **Deep RNN** (like in your notes), Step 2 would look like a tower.
   - **Layer 1:** Calculates "Low-level grammar" of "learns" + context of "AI".
   - **Layer 2:** Takes the result from Layer 1 and calculates "High-level meaning" (e.g., "This is a sentence about technology").
   - The "Vertical" axis is just stacking more processing on top of the current time step.
2. **The Signal Processing View:**
   - **Input Signal:** The sequence of word vectors ($[1, 0] \rightarrow [0, 1]$).
   - **IIR Filter:** Because $h_2$ contains a piece of $h_1$, the "energy" of the word "AI" is still present at Time 2. If we added 100 more words, a tiny fraction of "AI" would theoretically still be echoing in the state (Infinite Impulse Response).
3. **The Kalman View:**
   - At Time 2, we had a "Predicted State" (where we thought the sentence was going based on "AI") and a "Measurement" (the actual word "learns").
   - The RNN weights ($W_h, B$) acted as the transition matrices to merge these into the optimal current estimate $h_2$.

**Commented [14]:** I liked how Gemini included the Kalman filter view when walking through a toy example being fed into a simple RNN.




Great thanks this makes sense. I have a confusion about the part of the lecture notes I attached. Can you please explain this more clearly?

Show thinking

This image (from Page 5 of your notes) addresses a fundamental question in Deep Learning and Control Theory: **"If we can't see the hidden machinery (the state $x$), can we still learn how it works just by looking at the inputs and outputs?"**

Here is the breakdown of the specific handwritten notes and diagrams in that image.

**1. The Core Question**

The text at the top asks:

> *"Question: Can we learn $\tilde{A}, \tilde{B}, F$ for KF? Why or why not?"*

**Commented [15]:** I was a bit confused about the part in the notes that talked about how we can't learn the Kalman filter matrices with only inputs and outputs but no hidden states.

This asks: If we don't know the physics (the matrices), and we only have "traces" (recordings of inputs $u$ and noisy outputs $y$), can we perfectly reverse-engineer the "true" internal matrices of the system?

**2. The Answer: "No" (The Coordinate Problem)**

The notes answer:

> *"No. Can change coordinates for $x$ and this is unobservable from $Y$."*

This is the most confusing part, so let's use an analogy.

**The "Currency" Analogy:** Imagine a hidden bank account (the Hidden State $x$) that grows by 10% every year.

- **Scenario A:** The account is in **Dollars**. (Start: $100 → Next: $110).
- **Scenario B:** The account is in **Euros**. (Start: €90 → Next: €99).

If you are an observer on the outside, and you only see "The value increased by 10%," you **cannot** mathematically prove whether the internal bank account is holding Dollars or Euros. Both "coordinate systems" (Dollars vs. Euros) fit the data perfectly.

In the notes, this is what "Change coordinates for $x$" means. The network can learn to predict the *outcome*, but it might learn to represent the internal state in "Euros" when the true physics is actually in "Dollars."

**3. The "Good News": It Doesn't Matter!**

The text at the bottom says:

**Commented [16]:** Again, I think this analogy is a bit too unrelated to the topic and wasn't that helpful in understanding that part from the lecture notes.

> *"This basically works! We learn the essence of the KF but not the specific coordinate system for $\hat{x}$."*

Even though the network might use the "wrong" units internally (e.g., it learns a weird, rotated mathematical representation of the state), it **doesn't matter for prediction**.

- If the goal is just to predict the next output $y_{t+1}$ (Self-Supervision), the "fake" coordinate system works just as well as the "true" one.
- The diagram shows the network taking previous inputs $(u_t, y_t)$ and effectively learning to predict the next $y_{t+1}$ without ever being told the true $x$.

**4. The Practical Takeaway ("Self-Supervision")**

The notes conclude with a powerful practical point:

> *"Practically, this means that we can learn the coordinate system with just a little bit of $x$ data."*

This justifies how we train massive models (like GPT or modern RNNs):

1. **Phase 1 (Big Data):** Train on billions of examples using just inputs/outputs (Self-Supervision). The model learns the complex physics/grammar, but its internal "labels" might be weird/scrambled.
2. **Phase 2 (Small Data):** Use a tiny bit of labeled data ("Ground Truth $x$") to map the weird internal state to the human-readable state.

**Summary of the Image:** You **cannot** learn the exact "true" physics equations just from raw data because the internal units are ambiguous. However, you **can** learn a model that is *functionally equivalent* and predicts perfectly. This is the foundation of self-supervised learning.